

iOS 项目聊天功能代码优化

iOS 项目聊天功能代码优化建议

一、消息加载优化

1. 问题

当前代码一次性加载所有历史消息（默认 `MAX_LIMIT_NUMS = 200`），消息数量过多时会导致：

- 网络请求数据量大，加载缓慢；
- 列表渲染压力大，界面卡顿；
- 内存占用过高，可能引发性能问题。

2. 优化方案

改为 **分页加载**，每次仅加载 20-30 条消息，用户上拉列表时再加载更早的历史消息，降低单次加载压力。

3. 代码示例

```
// 1. 在 ZYXChatViewController.h 中添加分页参数
@interface ZYXChatViewController ()
@property (nonatomic, assign) NSInteger currentPage; // 当前页码（从1开始）
@property (nonatomic, assign) NSInteger pageSize; // 每页加载条数（建议20-30）
@property (nonatomic, assign) BOOL isLoadingMore; // 防止重复加载标记
@end

// 2. 初始化分页参数（viewDidLoad 中）
- (void)viewDidLoad {
    [super viewDidLoad];
    // 其他初始化代码...
    self.currentPage = 1; // 初始页码为1
    self.pageSize = 20; // 每页加载20条
```

```

    self.isLoadingMore = NO; // 初始无加载中
}
// 3. 请求历史消息时携带分页参数
- (void)kmessageListURLFunction {
    if (![Global sharedInstance] islogged) || self.isLoadingMore) {
        return;
    }
    self.isLoadingMore = YES; // 标记为加载中

    ZYUserDefaultsModel *userDefaultsModel = [ZYUserDefaultsModel userDefaultsModel];
    // 请求参数中添加分页信息
    NSDictionary *parameters = @{
        @"userId": userDefaultsModel.userId,
        @"employeeUuid": self.employeeUuid,
        @"messageType": @"1",
        @"page": @(self.currentPage), // 当前页码
        @"pageSize": @(self.pageSize) // 每页条数
    };

    WS(weakSelf);
    [ZYNetworkRequest postBeanNoPrompt:parameters
                     url:kmessageListURL
                     success:^(id json) {
        ZYNetWorkModel *object = (ZYNetWorkModel *)json;
        if ([object.status isEqualToString:@"1"]) {
            NSArray *newMessages = (NSArray *)object.data;
            if (newMessages.count > 0) {
                // 分页加载：更早的消息插入到数组头部（历史消息在上方）
                [weakSelf.chatContentArr insertObjects:newMessages atIndexes:[NSIndexSet
indexSetWithIndexesInRange:NSMakeRange(0, newMessages.count)]];
                weakSelf.currentPage++; // 页码+1，下次加载下一页
            } else {
                // 无更多消息（可提示用户）
                [MBProgressHUD showMessage:@"已加载全部历史消息" toView:weakSelf.view];
            }

            dispatch_async(dispatch_get_main_queue(), ^{
                [weakSelf.weChatTableView reloadData];
                weakSelf.isLoadingMore = NO; // 取消加载标记
            });
        }
    }
}

```

```

    } failure:^(NSError *error) {
        weakSelf.isLoadingMore = NO; // 失败时也取消加载标记
        // 错误处理...
    } progress:^(float progress) {}];
}
// 4. 列表上拉加载 (UITableViewDelegate 方法)
- (void)scrollViewDidScroll:(UIScrollView *)scrollView {
    // 计算滚动位置：列表顶部露出高度 > 100px 时触发加载更多
    CGFloat offsetY = scrollView.contentOffset.y;
    CGFloat contentHeight = scrollView.contentSize.height;
    CGFloat frameHeight = scrollView.frame.size.height;
    if (offsetY < -100 && !weakSelf.isLoadingMore) {
        [self kmessageListURLFunction]; // 加载更早的历史消息
    }
}
}

```

二、清理无用代码与资源

1. 问题

- 代码中存在大量注释代码（如废弃的请求逻辑、调试代码），影响可读性；
- 定时器（如聊天列表的 10 秒刷新定时器）未在页面销毁时清理，可能导致野指针或内存泄漏；
- 未移除通知监听，可能引发重复回调或崩溃。

2. 优化方案

- 删除所有无用注释代码、废弃逻辑，保持代码简洁；
- 在 `dealloc` 方法中清理定时器、通知监听，避免资源泄漏。

3. 代码示例

```

// 在 ZYMessageListViewController.m 中添加 dealloc 方法
- (void)dealloc {
    // 1. 清理定时器（停止并置空）
    if (self->timer) {

```

```
[self->timer invalidate];
self->timer = nil;
}

// 2. 移除所有通知监听（避免重复回调）
[[NSNotificationCenter defaultCenter] removeObserver:self];

// 3. 释放其他大对象（如数据数组，辅助内存回收）
self.array = nil;
}
```

三、表格视图性能优化（Cell 高度缓存）

1. 问题

当前每次渲染列表时，都会调用 `[WeChatTableViewCell cellHeightForModel:]` 重新计算 Cell 高度，而同一消息的高度固定，重复计算会浪费性能，导致列表滚动卡顿。

2. 优化方案

使用 **字典缓存 Cell 高度**，以“消息唯一标识（如发送时间 + 索引）”为 Key，高度为 Value，计算一次后复用结果。

3. 代码示例

```
// 1. 在 ZYXChatViewController.h 中添加高度缓存字典
@interface ZYXChatViewController ()
@property (nonatomic, strong) NSMutableDictionary *heightCache; // Cell 高度缓存
@end

// 2. 初始化缓存字典（viewDidLoad 中）
- (void)viewDidLoad {
    [super viewDidLoad];
    // 其他初始化代码...
    self.heightCache = [NSMutableDictionary dictionary]; // 初始化缓存
}

// 3. 计算 Cell 高度时优先读取缓存
```

```
- (CGFloat)tableView:(UITableView *)tableView heightForRowAtIndexPath:(NSIndexPath *)
indexPath {
    chatModel *model = self.chatContentArr[indexPath.row];

    // 3.1 生成唯一缓存 Key（发送时间+行索引，确保唯一性）
    NSString *cacheKey = [NSString stringWithFormat:@"% %@_%ld", model.sendTime ?: @"", (
long)indexPath.row];

    // 3.2 读取缓存：若存在则直接返回，无需重新计算
    NSNumber *cachedHeight = self.heightCache[cacheKey];
    if (cachedHeight) {
        return cachedHeight.floatValue;
    }

    // 3.3 计算高度并缓存（仅首次计算）
    CGFloat cellHeight = [WeChatTableViewCell cellHeightForModel:model.content];
    self.heightCache[cacheKey] = @(cellHeight); // 存入缓存

    return cellHeight;
}

// 4. 清空缓存（如消息更新时，避免缓存失效）
- (void)clearHeightCache {
    [self.heightCache removeAllObjects];
}
```

四、简化键盘处理（使用 IQKeyboardManager）

1. 问题

当前手动实现 `keyboardWillShow:`、`keyboardWillHide:` 等方法处理键盘适配，代码复杂且易出错（如不同设备键盘高度适配、输入框遮挡问题）。

2. 优化方案

项目已集成 **IQKeyboardManager**（iOS 主流键盘管理库），可直接启用，自动处理键盘弹出 / 收起时的界面调整，无需手动编写键盘逻辑。

3. 使用方式

```
// 在 AppDelegate.m 的 didFinishLaunchingWithOptions 中启用 IQKeyboardManager
#import "IQKeyboardManager.h"
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(
NSDictionary *)launchOptions {
    // 其他初始化代码...

    // 启用 IQKeyboardManager
    IQKeyboardManager *keyboardManager = [IQKeyboardManager sharedManager];
    keyboardManager.enable = YES;           // 全局启用
    keyboardManager.enableAutoToolbar = NO; // 禁用默认工具栏（如需自定义输入栏）
    keyboardManager.shouldResignOnTouchOutside = YES; // 点击空白区域收起键盘
    keyboardManager.keyboardDistanceFromTextField = 10; // 键盘与输入框间距（10px）

    return YES;
}
// 注意：删除原有的 keyboardWillShow:、keyboardWillHide: 等手动键盘处理代码，避免冲突
```

五、网络请求错误处理优化

1. 问题

当前发送消息、加载历史消息失败时，无任何用户提示（如“发送失败”“网络错误”），用户无法感知操作结果，体验较差。

2. 优化方案

在网络请求的 `failure` 回调中，添加 **用户提示**（如 HUD 弹窗），告知失败原因，并提供重试入口（可选）。

3. 代码示例

```
// 发送消息请求的错误处理优化
- (void)sendFunction:(NSString *)sendStr textView:(UITextView *)textView {
```

```

ZYUserDefaultsModel *userDefaultsModel = [ZYUserDefaultsModel standardUserDefaults];
NSDictionary *parameters = @{
    @"userId": userDefaultsModel.userId,
    @"content": sendStr,
    @"chatType": @"1",
    @"employeeUuid": self.employeeUuid,
};

// 显示加载中 HUD
[MBProgressHUD showLoading:@"发送中..." toView:self.view];

WS(weakSelf);
[ZYNetworkRequest postBeanNoPrompt:parameters
    url:kseURL
    success:^(id json) {
    [MBProgressHUD hideHUDForView:weakSelf.view]; // 隐藏加载HUD
    ZYNetWorkModel *object = (ZYNetWorkModel *)json;
    if ([object.status isEqualToString:@"1"]) {
        // 成功逻辑...
        [MBProgressHUD showSuccess:@"发送成功" toView:weakSelf.view];
    } else {
        // 业务错误（如参数错误）
        [MBProgressHUD showError:object.message ? @"发送失败" toView:weakSelf.view];
    }
} failure:^(NSError *error) {
    [MBProgressHUD hideHUDForView:weakSelf.view]; // 隐藏加载HUD

    // 网络错误处理（区分错误类型）
    NSString *errorMsg = @"发送失败，请检查网络后重试";
    if (error.code == NSErrorURLErrorTimedOut) {
        errorMsg = @"请求超时，请重试";
    } else if (error.code == NSErrorURLErrorNotConnectedToInternet) {
        errorMsg = @"无网络连接，请检查网络设置";
    }

    // 显示错误提示
    [MBProgressHUD showError:errorMsg toView:weakSelf.view];

    // 可选：添加重试按钮
    dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(1.5 * NSEC_PER_SEC)),
dispatch_get_main_queue(), ^{

```

```
UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"提示" message:
errorMsg preferredStyle:UIAlertControllerStyleAlert];
[alert addAction:[UIAlertAction actionWithTitle:@"重试" style:UIAlertActionStyleDefault
handler:^(UIAlertAction * _Nonnull action) {
    [weakSelf sendFunction:sendStr textView:textView]; // 重试发送
}]];
[alert addAction:[UIAlertAction actionWithTitle:@"取消" style:UIAlertActionStyleCancel
handler:nil]];
[weakSelf presentViewController:alert animated:YES completion:nil];
});
} progresss:^(float progress) {}];
}
```

总结

以上优化方案均为“低侵入、高收益”类型，无需重构核心逻辑即可实现：

1. 分页加载减少网络与渲染压力，提升列表流畅度；
2. 资源清理避免内存泄漏，增强稳定性；
3. 高度缓存降低重复计算，优化滚动性能；
4. IQKeyboardManager 简化键盘处理，减少代码量；
5. 错误提示提升用户体验，减少操作困惑。

优化后，聊天功能的性能、稳定性与用户体验将得到显著提升。

（注：文档部分内容可能由 AI 生成）